Cyberscope

A TAC Security Company

Audit Report

Hero Arena Play

June 2026

Repository	https://github.com/HaGameFi/HeroArena_contracts
Commit	7c938f9b127ddfb2e26842441d0f7acd98c22a29
Audited by	© cyberscope

Table of Contents

Risk Classification

The criticality of findings in Cyberscope's smart contract audits is determined by evaluating multiple variables. The two primary variables are:

1. **Likelihood of Exploitation**: This considers how easily an attack can be executed, including the economic feasibility for an attacker.
2. **Impact of Exploitation**: This assesses the potential consequences of an attack, particularly in terms of the loss of funds or disruption to the contract's functionality.

Based on these variables, findings are categorized into the following severity levels:

1. **Critical**: Indicates a vulnerability that is both highly likely to be exploited and can result in significant fund loss or severe disruption. Immediate action is required to address these issues.
2. **Medium**: Refers to vulnerabilities that are either less likely to be exploited or would have a moderate impact if exploited. These issues should be addressed in due course to ensure overall contract security.
3. **Minor**: Involves vulnerabilities that are unlikely to be exploited and would have a minor impact. These findings should still be considered for resolution to maintain best practices in security.
4. **Informative**: Points out potential improvements or informational notes that do not pose an immediate risk. Addressing these can enhance the overall quality and robustness of the contract.

Severity	Likelihood / Impact of Exploitation
Critical	Highly Likely / High Impact
Medium	Less Likely / High Impact or Highly Likely/ Lower Impact
Minor / Informative	Unlikely / Low to no Impact

Review

Audit Updates

Initial Audit	11 May 2026 https://github.com/cyberscope-io/audits/blob/main/hap/v1/audit.pdf
Corrected Phase 2	02 June 2026

Source Files

Filename	SHA256
HeroArenaBattle.sol	208a376412be2348d9923a15de5554ef5d7fe763f61c881fa474aea5e5fafd72
HapVesting.sol	27ed800961fdf457f7f8cf030e75c50a9ef12bd7eb0cc81deb39c269e9e96232
HapTreasury.sol	76b5ce187dd119da28aaed5f6140793bbb3c683cb11b933a6e7658f2be627ff5
HapToken.sol	102d88b958d5102d7516238c3a77316cf1a15772e2e53b55427df52c6e8529aa
HeroArenaProfileInterface.sol	0a1e71700b06227920678296d14436fef58e7910212dd77812b29dbfbe1072b2
HeroArenaProfile.sol	a3db8c302939f3faac5cfe1c580cb057254f7577a66c4ece334e7939b63a33fc
HeroArenaMiningFactoryV1.sol	2ba06415d923032ef4c08dd3067a87396f283a6715b1a48f78895b7d07b4f61f

HeroArenaMeetTheCouncil.sol	a5ea97f970d912503831994a5509e1e6e2c9d2f901fe2db7b7049604d82a882a
HeroArenaChallenges.sol	e5f988056b613b0408fdc4fabb296e7e1c892a487f802bf865ab6ca0399fc1bf
HeroArenaAvatars.sol	dc097cb9082822e7ad24ad6e96170dd7c40080ffa9404e5ad1dee81ede995197

Overview

The `Hero Arena Play` ecosystem is built from nine contracts that together cover token issuance, controlled fund distribution, gameplay, cosmetic ownership, identity, and progression. The token and treasury layer handles the supply and governed release of funds, the gameplay layer manages battles and PvE challenges, the NFT layer mints and tracks avatar collectibles, and the profile layer ties players to teams, cosmetics, and points. The contracts interoperate through shared roles and cross-contract calls to form a single connected system.

HapToken.sol

`HapToken` is the fixed-supply ERC20 token for the Hero Arena ecosystem. It mints the full `1,000,000,000 HAP` supply to the initial admin at deployment and does not include any further minting functionality. The contract extends burnable and pausable ERC20 behavior, tracks cumulative burns through `totalBurned`, and uses role-based access control for pausing and blacklist administration. It also includes protected addresses that cannot be blacklisted and an admin-only recovery function for ERC20 tokens accidentally sent to the token contract, excluding HAP itself.

HapVesting.sol

`HapVesting` manages HAP vesting schedules for beneficiaries. Each schedule supports a TGE unlock amount, cliff duration, linear vesting duration, release tracking, and optional revocation for revocable allocations such as team or advisor grants. The contract allows admins to create multiple schedules per beneficiary, lets anyone release vested tokens for a schedule, and provides a batch `releaseAllMine()` flow for beneficiaries. It also tracks aggregate allocated and released amounts, supports revocation with auto-release of vested tokens, and exposes view helpers for frontend schedule and vesting calculations.

HapTreasury.sol

`HapTreasury` is the project treasury manager for native BNB and ERC20 assets. It records received funds, tracks cumulative spending, and disburses assets through a proposal lifecycle: creation by `PROPOSAL_ROLE`, approval by the admin role, a seven-day timelock, and execution by `EXECUTOR_ROLE` before expiry. It includes cancellation, status, and timing views, plus guardian-controlled emergency pause functionality. When paused, the admin can perform emergency withdrawals, while normal proposal approval and execution are disabled.

HeroArenaBattle.sol

`HeroArenaBattle` manages wager-based battles between registered Hero Arena players. Users can create battles with an allowed bet token, join open or targeted battles, and have results settled by the `LIQUIDATOR_ROLE`. The contract tracks outstanding locked bets, applies optional protocol fees, pays the winner, and can optionally distribute a configured bonus token. It also includes owner-controlled settings for allowed tokens, minimum bet amounts, battle availability, forbidden players, protocol fee configuration, bonus configuration, fee withdrawal, token deposits, and rescue of funds not reserved for active bets or accrued fees.

HeroArenaAvatars.sol

`HeroArenaAvatars` is an enumerable ERC721 collection for Hero Arena avatar NFTs. The owner can mint NFTs to users, assign each token an `avatarId`, burn NFTs, and set metadata labels and creation timestamps for each avatar type. The contract tracks minted and burned counts per avatar ID, exposes batch lookups for avatar IDs and avatar metadata, and allows users or external consumers to enumerate all avatar token IDs owned by an address.

HeroArenaProfile.sol

`HeroArenaProfile` manages user profiles that are paid for in HAP. Users can register once, choose a joinable team, and later attach approved avatar or frame ERC721 tokens to their profile by transferring those NFTs into the contract. The owner manages teams, registration and update fees, and approved avatar or frame collections, while role-gated accounts can change user teams or adjust user and team points. The contract therefore acts as the identity, team, cosmetic, and points registry for Hero Arena users.

HeroArenaMiningFactoryV1.sol

`HeroArenaMiningFactoryV1` is the minting factory for Hero Arena avatar NFTs. At deployment it creates a new `HeroArenaAvatars` contract, initializes a fixed list of avatar names, stores the HAP mint price, and controls whether public claiming is available. Users can mint an avatar by paying the configured HAP price, while the owner can update claim availability, change the NFT price, withdraw collected HAP, and transfer ownership of the underlying avatar contract through a two-step pending-owner flow.

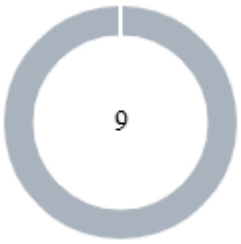
HeroArenaChallenges.sol

`HeroArenaChallenges` tracks PvE challenge completions by user and level. A challenge admin can submit a user's completion for a level, ensuring the same user can only submit each level once, and the contract records a generated challenge ID mapped to the submitted level ID. It also stores level names and reward-point values, exposes reward and submission-status views, and provides batch lookup helpers for level IDs and level metadata.

HeroArenaMeetTheCouncil.sol

`HeroArenaMeetTheCouncil` connects the challenge registry with the profile points system. The owner can initialize a fixed set of council levels and toggle whether submissions are available, while addresses with `OPERATOR_ROLE` can submit a user's completed level. On submission, the contract records the challenge through `HeroArenaChallenges`, reads the level's reward points, and forwards those points to `HeroArenaProfile` for the user.

Findings Breakdown



Critical	0
Medium	0
Minor / Informative	9

Severity	Unresolved	Acknowledged	Resolved	Other
Critical	0	0	0	0
Medium	0	0	0	0
Minor / Informative	9	0	0	0

Diagnostics

Critical Medium Minor / Informative

Severity	Code	Description	Status
●	ST	Stops Transactions	Unresolved
●	BC	Blacklists Addresses	Unresolved
●	MT	Mints Tokens	Unresolved
●	BT	Burns Tokens	Unresolved
●	CCR	Contract Centralization Risk	Unresolved
●	ENC	External NFT Custody	Unresolved
●	MM	Multi-Authority Maintenance	Unresolved
●	DRG	Detach Reentrancy Gap	Unresolved
●	AME	Address Manipulation Exploit	Unresolved

ST - Stops Transactions

Criticality	Minor / Informative
Location	contracts/HapToken.sol#L164,L205
Status	Unresolved

Description

The contract owner has the authority to stop the sales for all users. The owner may take advantage of it by calling `pause()`. Additionally, the sales can be stopped by blacklisting the pair contract.

```
function pause() external onlyRole(PAUSER_ROLE) {
    _pause();
}
...
function _update(
    address from,
    address to,
    uint256 value
) internal override(ERC20, ERC20Pausable) {
    // Blacklist check
    if (blacklisted[from]) revert AddressIsBlacklisted(from);
    if (blacklisted[to]) revert AddressIsBlacklisted(to);

    // Track all burns uniformly: burn(), burnFrom(), and
burnFromRevenue() all pass through here
    if (to == address(0)) {
        totalBurned += value;
    }

    super._update(from, to, value);
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

Renouncing the ownership, which will eliminate the threats but it is non-reversible.

BC - Blacklists Addresses

Criticality	Minor / Informative
Location	contracts/HapToken.sol#L131
Status	Unresolved

Description

The contract owner has the authority to stop addresses from transactions. The owner may take advantage of it by calling the `blacklist` function.

```
function blacklist(address account) external onlyRole(BLACKLIST_ROLE) {
    if (account == address(0)) revert CannotBlacklistZeroAddress();
    if (protectedFromBlacklist[account]) revert
CannotBlacklistProtectedAddress(account);
    if (blacklisted[account]) revert AddressAlreadyBlacklisted(account);
    blacklisted[account] = true;
    emit AddressBlacklisted(account, msg.sender);
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

Temporary Solutions:

These measurements do not decrease the severity of the finding

- Introduce a time-locker mechanism with a reasonable delay.
- Introduce a multi-signature wallet so that many addresses will confirm the action.
- Introduce a governance model where users will vote about the actions.

Permanent Solution:

- Renouncing the ownership, which will eliminate the threats but it is non-reversible.

MT - Mints Tokens

Criticality	Minor / Informative
Location	contracts/HeroArenaAvatars.sol#L41
Status	Unresolved

Description

The contract owner has the authority to mint tokens. The owner may take advantage of it by calling the `mint` function. As a result, the contract tokens will be highly inflated.

```
function mint(address _to, uint8 _avatarId) external onlyOwner returns
(uint256) {
    _tokenCounter += 1;
    uint256 _newTokenId = _tokenCounter;
    _avatarIds[_newTokenId] = _avatarId;
    avatarCount[_avatarId] += 1;
    _mint(_to, _newTokenId);
    return _newTokenId;
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

BT - Burns Tokens

Criticality	Minor / Informative
Location	contracts/HeroArenaAvatars.sol#L61
Status	Unresolved

Description

The contract owner has the authority to burn tokens from a specific address. The owner may take advantage of it by calling the `burn` function. As a result, the targeted address will lose the corresponding tokens.

```
function burn(uint256 _tokenId) external onlyOwner {
    uint8 _avatarId = _avatarIds[_tokenId];
    avatarCount[_avatarId] -= 1;
    avatarBurnCount[_avatarId] += 1;
    delete _avatarIds[_tokenId];
    _burn(_tokenId);
}
```

Recommendation

The team should carefully manage the private keys of the owner's account. We strongly recommend a powerful security mechanism that will prevent a single user from accessing the contract admin functions.

CCR - Contract Centralization Risk

Criticality	Minor / Informative
Location	contracts/HapTreasury.sol contracts/HapToken.sol contracts/HapVesting.sol contracts/HeroArenaBattle.sol
Status	Unresolved

Description

The contract's functionality and behavior are heavily dependent on external parameters or configurations. While external configuration can offer flexibility, it also poses several centralization risks that warrant attention. Centralization risks arising from the dependence on external configuration include Single Point of Control, Vulnerability to Attacks, Operational Delays, Trust Dependencies, and Decentralization Erosion.

Recommendation

To address this finding and mitigate centralization risks, it is recommended to evaluate the feasibility of migrating critical configurations and functionality into the contract's codebase itself. This approach would reduce external dependencies and enhance the contract's self-sufficiency. It is essential to carefully weigh the trade-offs between external configuration flexibility and the risks associated with centralization.

ENC - External NFT Custody

Criticality	Minor / Informative
Location	contracts/HeroArenaProfile.sol#L220,249
Status	Unresolved

Description

`updateAvatar()` and `updateFrame()` accept NFT contract addresses supplied by the user, as long as those addresses have been granted `AVATAR_ROLE` or `FRAME_ROLE`. The contract then calls `ownerOf()` and `safeTransferFrom()` on the supplied NFT contract and stores the NFT in `HeroArenaProfile`.

Because the NFT contract controls the behavior of these external calls, unsafe or non-standard whitelisted NFTs can cause profile updates to revert, behave unexpectedly, or create custody and compatibility issues. The whitelist is therefore security-critical and should only include NFT contracts whose transfer behavior and ERC721 compliance are trusted.

```
function updateAvatar(address _avatarAddress, uint256 _tokenId) external
{
    require(hasRegistered[msg.sender], "User not registered");
    require(hasRole(AVATAR_ROLE, _avatarAddress), "Avatar address
invalid");

    IERC721 _avatarToken = IERC721(_avatarAddress);
    require(msg.sender == _avatarToken.ownerOf(_tokenId), "Only owner can
transfer his/her NFT");

    _userMapping[msg.sender].avatarAddress = _avatarAddress;
    _userMapping[msg.sender].tokenId = _tokenId;

    _avatarToken.safeTransferFrom(msg.sender, address(this), _tokenId);
    ...
}

function addAvatarAddress(address _avatarAddress) external onlyOwner {
    require(IERC721(_avatarAddress).supportsInterface(0x80ac58cd), "Not
ERC721");
    _grantRole(AVATAR_ROLE, _avatarAddress);
}
```

Recommendation

Treat avatar and frame whitelisting as a trusted integration process. Only approve NFT contracts that have been reviewed for ERC721 compliance, predictable transfer behavior, and safe interaction with profile custody. Consider documenting whitelist criteria and adding operational checks before granting avatar or frame roles.

MM - Multi-Authority Maintenance

Criticality	Minor / Informative
Location	contracts/HeroArenaMeetTheCouncil.sol contracts/HeroArenaChallenges.sol contracts/HeroArenaProfile.sol contracts/HeroArenaMiningFactoryV1.sol contracts/HapToken.sol contracts/HapVesting.sol contracts/HapTreasury.sol
Status	Unresolved

Description

The system uses multiple authority models across contracts, including `Ownable`, `AccessControl`, role-gated operators, owner-gated factories, and cross-contract role dependencies. Several flows depend on manually maintained permissions, such as granting `CHALLENGE_ADMIN_ROLE` to the council contract, assigning profile point and operator roles, and preserving ownership over deployed avatar contracts.

This creates operational fragility because contracts may deploy successfully but remain partially non-functional until the correct role and ownership wiring is completed. Future ownership transfers, role revocations, or missed deployment steps can break core flows even when the underlying contracts are otherwise working.

```
contract HeroArenaProfile is AccessControl, ERC721Holder, Ownable {
    bytes32 public constant AVATAR_ROLE = keccak256("AVATAR_ROLE");
    bytes32 public constant FRAME_ROLE = keccak256("FRAME_ROLE");
    bytes32 public constant POINT_ROLE = keccak256("POINT_ROLE");
    bytes32 public constant SPECIAL_ROLE = keccak256("SPECIAL_ROLE");

    constructor(...) Ownable(msg.sender) {
        ...
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }
}

contract HeroArenaMeetTheCouncil is AccessControl, Ownable {
    bytes32 public constant OPERATOR_ROLE = keccak256("OPERATOR_ROLE");

    constructor(...) Ownable(msg.sender) {
        ...
        _grantRole(DEFAULT_ADMIN_ROLE, msg.sender);
    }

    function submitLv(address _userAddress, uint8 _lvId) external
onlyOperator {
        uint256 _challengeId = HeroArenaChallengesSC.submit(_userAddress,
_lvId);
        HeroArenaProfileSC.increaseUserPoints(_userAddress,
_lvRewardPoints, _lvId);
    }
}

contract HeroArenaChallenges is AccessControl {
    bytes32 public constant CHALLENGE_ADMIN_ROLE =
keccak256("CHALLENGE_ADMIN_ROLE");

    modifier onlyChallengeAdmin() {
        require(hasRole(CHALLENGE_ADMIN_ROLE, msg.sender), "Not a
challenge admin role");
        _;
    }
}
```

Recommendation

Define and document a single authority model or a complete role-management runbook for the system. Deployment and ownership-transfer procedures should atomically assign required roles, revoke stale authorities, and verify cross-contract permissions before contracts are considered operational. Where possible, reduce manual wiring by granting required roles during construction or through explicit initialization functions.

DRG - Detach Reentrancy Gap

Criticality	Minor / Informative
Location	HeroArenaProfile#L358,378
Status	Unresolved

Description

The `detachAvatar()` and `detachFrame()` functions allow registered users to recover their currently-assigned avatar or frame NFT without supplying a replacement, and both correctly follow the Checks-Effects-Interactions pattern by clearing the user's profile slot in state before calling `safeTransferFrom` to return the NFT. However, neither function is protected by a `nonReentrant` modifier, and the `HeroArenaProfile` contract does not inherit `ReentrancyGuard` at all. Because the NFT contract whose `safeTransferFrom` is invoked is one the user previously attached through `updateAvatar()` or `updateFrame()`, the callback executes code that originates from an externally-controlled, role-whitelisted contract. While the prior state clearing prevents a direct double-detach of the same slot, it does not prevent a malicious or non-standard whitelisted NFT from reentering other entry points on `HeroArenaProfile` during the callback, such as `updateAvatar`, `updateFrame`, or any role-gated flow accessible to the caller, before the original detach has fully concluded. This approach departs from the defensive custody and payout patterns used elsewhere in the protocol and places the entire safety boundary on the off-chain whitelisting process for every ERC721 contract granted `AVATAR_ROLE` or `FRAME_ROLE`.

```
function detachAvatar() external
function detachFrame() external
```

Recommendation

Inherit OpenZeppelin's `ReentrancyGuard` in `HeroArenaProfile` and apply the `nonReentrant` modifier to `detachAvatar()` and `detachFrame()`, and consider extending the same protection to `updateAvatar()` and `updateFrame()` since they perform analogous external `safeTransferFrom` calls into user-supplied NFT contracts.

AME - Address Manipulation Exploit

Criticality	Minor / Informative
Location	HeroArenaBattle.sol#L577
Status	Unresolved

Description

The contract's design includes functions that accept external contract addresses as parameters without performing adequate validation or authenticity checks. This lack of verification introduces a significant security risk, as input addresses could be controlled by attackers and point to malicious contracts. Such vulnerabilities could enable attackers to exploit these functions, potentially leading to unauthorized actions or the execution of malicious code under the guise of legitimate operations.

The `_tryPayout()` function attempts to determine whether an ERC20 transfer was successful by requiring the returned value to be exactly 1. While this behavior matches many standard ERC20 implementations, some non-standard tokens may successfully execute the transfer while returning a different non-zero value. In such cases, the token transfer would succeed, but the function would incorrectly report failure. This could cause the protocol to trigger fallback accounting mechanisms such as pending payouts despite the recipient already having received the tokens, resulting in inconsistent accounting and potentially duplicate payouts.

```
if (ret.length == 0) {
    ok = true;
} else if (ret.length == 32) {
    ok = (abi.decode(ret, (uint256)) == 1);
} else {
    ok = false;
}
```

Recommendation

To mitigate this risk and enhance the contract's security posture, it is imperative to incorporate comprehensive validation mechanisms for any external contract addresses

passed as parameters to functions. This could include checks against a whitelist of approved addresses, verification that the address implements a specific contract interface or other methods that confirm the legitimacy and integrity of the external contract. Implementing such validations helps prevent malicious exploits and ensures that only trusted contracts can interact with sensitive functions.

Functions Analysis

Contract	Type	Bases		
	Function Name	**Visibility**	**Mutability**	**Modifiers**
HeroArenaProfile	Implementation	AccessControl, ERC721Holder, Ownable		
		Public	✓	Ownable
	_transferOwnership	Internal	✓	
	addTeam	External	✓	onlyOwner
	getTeam	External		-
	renameTeam	External	✓	onlyOwner
	makeTeamJoinable	External	✓	onlyOwner
	makeTeamNotJoinable	External	✓	onlyOwner
	claimFee	External	✓	onlyOwner
	updateFeeCost	External	✓	onlyOwner
	createProfile	External	✓	-
	updateAvatar	External	✓	-
	updateFrame	External	✓	-
	detachAvatar	External	✓	-
	detachFrame	External	✓	-
	addAvatarAddress	External	✓	onlyOwner

	addFrameAddress	External	✓	onlyOwner
	changeTeam	External	✓	onlySpecial
	increaseUserPoints	External	✓	onlyPoint
	increaseUserPointsBatch	External	✓	onlyPoint
	increaseTeamPoints	External	✓	onlyPoint
	decreaseUserPoints	External	✓	onlyPoint
	decreaseUserPointsBatch	External	✓	onlyPoint
	decreaseTeamPoints	External	✓	onlyPoint
	getUserProfile	External		-
HeroArenaMiningFactoryV1	Implementation	Ownable		
		Public	✓	Ownable
	_transferOwnership	Internal	✓	
	updateAvailableClaim	External	✓	onlyOwner
	mintNFT	External	✓	-
	updateNFTPrice	External	✓	onlyOwner
	proposeNFTContractOwnership	External	✓	onlyOwner
	cancelNFTContractOwnership	External	✓	onlyOwner
	acceptNFTContractOwnership	External	✓	-
	claimFee	External	✓	onlyOwner

HeroArenaMeetTheCouncil	Implementation	AccessControl, Ownable		
		Public	✓	Ownable
	_transferOwnership	Internal	✓	
	initLevels	External	✓	onlyOwner
	updateAvailableSubmit	External	✓	onlyOwner
	submitLv	External	✓	onlyOperator
HeroArenaChallenges	Implementation	AccessControl		
		Public	✓	-
	submit	External	✓	onlyChallengeAdmin
	setLevelNameAndRewardPoints	External	✓	onlyChallengeAdmin
	challengeExists	External		-
	getLevelRewardPoints	External		-
	getSubmitStatus	External		-
	getLevelIdBatch	External		-
	getLevelNameAndPointsBatch	External		-
HeroArenaBattle	Implementation	Ownable, AccessControl, ReentrancyGuard		
		Public	✓	Ownable

		External	Payable	-
	createBattle	External	Payable	nonReentrant onlyRegisteredAndAllowed
	joinExistBattle	External	Payable	nonReentrant onlyRegisteredAndAllowed
	settleBattle	External	✓	nonReentrant onlyRole
	closeBattle	External	✓	nonReentrant onlyRole
	claimPayout	External	✓	nonReentrant
	getBattleInfo	External		-
	getBattleCount	External		-
	updateAllowedBetToken	External	✓	onlyOwner
	updateAvailableCreateBattle	External	✓	onlyOwner
	updateForbiddenToPlay	External	✓	onlyOwner
	updateMinimumBetTokenAmount	External	✓	onlyOwner
	updateBonusToken	External	✓	onlyOwner
	setProtocolFee	External	✓	onlyOwner
	setProtocolFeeRecipient	External	✓	onlyOwner
	withdrawProtocolFees	External	✓	onlyOwner nonReentrant
	depositToken	External	✓	onlyOwner
	rescueExtraTokens	External	✓	onlyOwner nonReentrant
	_receiveBet	Internal	✓	

	_payout	Internal	✓	
	_hasFreeBalance	Internal		
	_tryPayout	Internal	✓	
HeroArenaAvatars	Implementation	ERC721Enumerable, Ownable		
		Public	✓	ERC721 Ownable
	supportsInterface	Public		-
	_baseURI	Internal		
	mint	External	✓	onlyOwner
	setAvatarNameAndCreatedTimestamp	External	✓	onlyOwner
	burn	External	✓	onlyOwner
	tokenExists	External		-
	getAvatarIdBatch	External		-
	getAvatarNameAndCreatedTimestampBatch	External		-
	getTokensByOwner	External		-
HapVesting	Implementation	AccessControl, ReentrancyGuard		
		Public	✓	-
	createVestingSchedule	External	✓	onlyRole

	release	External	✓	nonReentrant
	releaseAllMine	External	✓	nonReentrant
	revoke	External	✓	onlyRole nonReentrant
	rescueRevokedTokens	External	✓	onlyRole
	computeReleasableAmount	External		-
	computeVestedAmount	External		-
	computeReleasableForBeneficiary	External		-
	scheduleCount	External		-
	beneficiaryCount	External		-
	getSchedulesOf	External		-
	getActiveSchedulesOf	External		-
	activeScheduleCount	External		-
	getSchedule	External		-
	computeVestedAt	External		-
	_removeFromActive	Internal	✓	
	_computeReleasableAmount	Internal		
	_computeVestedAmount	Internal		
	_computeVestedAtTimestamp	Internal		
HapTreasury	Implementation	AccessControl, ReentrancyGuard, Pausable		

		Public	✓	-
	receiveFunds	External	✓	nonReentrant whenNotPaused
	depositBNB	External	Payable	whenNotPaused
		External	Payable	whenNotPaused
	createProposal	External	✓	onlyRole whenNotPaused
	approveProposal	External	✓	onlyRole whenNotPaused
	executeProposal	External	✓	onlyRole nonReentrant whenNotPaused
	cancelProposal	External	✓	-
	emergencyPause	External	✓	onlyRole
	unpause	External	✓	onlyRole
	emergencyWithdraw	External	✓	onlyRole whenPaused nonReentrant
	balanceOf	External		-
	tokenStats	External		-
	proposalStatus	External		-
	timeUntilExecutable	External		-

HapToken	Implementation	ERC20, ERC20Burnable, ERC20Pausable, AccessControl, ReentrancyGuard		
		Public	✓	ERC20
	setProtected	External	✓	onlyRole
	blacklist	External	✓	onlyRole
	unblacklist	External	✓	onlyRole
	pause	External	✓	onlyRole
	unpause	External	✓	onlyRole
	burnFromRevenue	External	✓	nonReentrant
	_update	Internal	✓	
	transferFrom	Public	✓	-
	burnFrom	Public	✓	-
	rescueERC20	External	✓	onlyRole
	burnStats	External		-

Inheritance Graph

A link to Hero Arena Play Inheritance Graph can be found in the following link

https://drive.google.com/file/d/1aK1MFs25h30tYvu2mw5QdCLh8q2qsPdk/view?usp=drive_link

Flow Graph

A link to Hero Arena Play Flow Graph can be found in the following link

https://drive.google.com/file/d/1XbHJJAOEfvD5sGKSEz6TKgnbfsJPrP6f/view?usp=drive_link

Summary

Hero Arena contracts implement a token ecosystem built around the fixed-supply HAP token, scheduled vesting distributions, treasury fund management, and wager-based player battles with optional bonus rewards. This audit investigates security issues, business logic concerns and potential improvements.

Disclaimer

The information provided in this report does not constitute investment, financial or trading advice and you should not treat any of the document's content as such. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes nor may copies be delivered to any other person other than the Company without Cyberscope's prior written consent. This report is not nor should be considered an "endorsement" or "disapproval" of any particular project or team. This report is not nor should be regarded as an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Cyberscope to perform a security assessment. This document does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors' business, business model or legal compliance. This report should not be used in any way to make decisions around investment or involvement with any particular project. This report represents an extensive assessment process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk Cyberscope's position is that each company and individual are responsible for their own due diligence and continuous security Cyberscope's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies and in no way claims any guarantee of security or functionality of the technology we agree to analyze. The assessment services provided by Cyberscope are subject to dependencies and are under continuing development. You agree that your access and/or use including but not limited to any services reports and materials will be at your sole risk on an as-is where-is and as-available basis Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives false negatives and other unpredictable results. The services may access and depend upon multiple layers of third parties.

About Cyberscope

Cyberscope is a TAC blockchain cybersecurity company that was founded with the vision to make web3.0 a safer place for investors and developers. Since its launch, it has worked with thousands of projects and is estimated to have secured tens of millions of investors' funds.

Cyberscope is one of the leading smart contract audit firms in the crypto space and has built a high-profile network of clients and partners.

A **TAC Security** Company

The Cyberscope team

cyberscope.io